

It is also possible to call the function `preg_match()` with three parameters as `preg_match($pat, $str, $val)` where the array variable `$val` contains the matched string. Consider the PHP script `regex2.php` shown below:

```
<?php
    $pat = '/b+/';
    $str = 'aaaabbbbbaaaa';
    if(preg_match($pat,$str,$val))
    {
        $temp = $val[0];
        echo "<b> Matched string is $temp </b>";
    }
    else
    {
        echo 'No Match Found';
    }
?>
```

To view the output of this script, open a Web browser and type 'localhost/regex2.php' on the address bar. The message 'Matched string is bbbb' will be displayed on the Web browser in bold text. This also tells us that the function `preg_match()` goes for a greedy match, which results in the longest possible match. Thus, the function does not match strings `b`, `bb`, or `bbb`; instead `bbbb` is the matched string. The variable `$val[0]` contains the entire text matched by the regular expression pattern. At this point, I should also mention the difference between strings inside single quotes and double quotes in PHP. The former are treated literally, whereas for the strings inside double quotes, the content of the variable is printed instead of just printing their names.

Other functions in preg

There are many other useful functions offered by the `preg` class of functions in PHP for regular expression processing other than the function `preg_match()`. But we will only discuss a very useful function called `preg_replace()` which replaces the matched string with another string. The function can be used with three parameters as follows: `preg_replace($pat, $rep, $str)` where `$pat` contains the regular expression pattern, `$rep` contains the replacement string, and `$str` contains the string to be searched for a pattern. Consider the PHP script `regex3.php` shown below:

```
<?php
    $pat = '/World/';
    $rep = 'Friends';
    $str = 'Hello World';
    if(preg_match($pat,$str))
    {
        $str = preg_replace($pat,$rep,$str);
        echo "<b> The modified string: $str </b>";
    }
}
```

```
}
else
{
    echo 'No Match Found';
}
?>
```

The function `preg_replace()` will not modify the contents of the variable `$str` as such. Instead the function will only return the modified string. In this example, the line of code '`$str = preg_replace($pat,$rep,$str);`' replaces the word 'World' with the word 'Friends', and this modified string is explicitly stored in the variable `$str`. To view the output of this script, open a Web browser and type `localhost/regex3.php` on the address bar. The message 'The modified string: Hello Friends' will be displayed on the Web browser in bold text. In case of both `regex2.php` and `regex3.php`, I have only shown the PHP portion of the scripts for want of space, but the complete scripts are available for download.

A regular expression for validating numbers

Now we are going to look at how our knowledge of regular expressions will help us validate numbers using PHP.

The aim is to check whether the given number entered through a text box in an HTML page is an integer or a real number, and print the same on the Web page in bold text. If the input text is neither an integer nor a real number, then the message 'Not a number' is displayed on the Web page in bold text. But remember, this statement is factually incorrect as mathematicians will be eager to point out that the input text could still be a number by being an irrational number like Π (Pi) or a complex number like $5 + 10i$. It could even be a quaternion or an octonion, even more bizarre number systems. But I think as far as practising computer science people are concerned, integers and real numbers are sufficient most of the times. To achieve this, we have two scripts called `number.html` and `number.php`. The script `number.html` is shown below:

```
<html>
<body>
    <form action="number.php" method="post">
        Enter a Number:
        <input type="text" name="number">
        <input type="submit" value="CLICK">
    </form>
</body>
</html>
```

The script `number.html` reads the number in a text field, and when the Submit button is pressed the script `number.php` is invoked. The input data is then passed to the script `number.php` by using the POST method for further processing. The script